

ABRA — Full Project Handoff

Written 2026-07-24. Everything a new session needs to work on this project without re-deriving it. Read section 1 before touching anything.

1. IMMEDIATE: THERE IS DAMAGE TO REPAIR

A run of `push-all.bat` (the version using `git merge -X ours`) broke two things:

Problem	Current state	Should be
<code>data/games.ladder.jsonl</code>	16,139 lines, 8,139 duplicates	7,547 unique
ABRA git	detached HEAD	on <code>main</code>

Nothing is lost — every game is present, just duplicated. Repair, in order:

```
git status                # look first
git checkout main
python engine/dedupe_store.py --write # 16,139 -> 7,547
python engine/sanity_check.py        # must be 96 passed, 0 failed
git add -A && git commit -m "repair: dedupe store after merge -X ours"
git push origin main
```

Then rewrite `push-all.bat`. It still contains `git merge -X ours`, which is the confirmed cause of store duplication three separate times (7,040 lines, then 401, then 8,139). `games.ladder.jsonl` is an APPEND-ONLY log; merging divergent histories replays the appended block. The replacement must: use `git rebase`, run `dedupe_store.py --write` before committing, and **stop on conflict** rather than resolving with `-X ours` (which silently discards the other side).

2. What this project is

ABRA — the Automated Battle Replay Analyzer. It ingests public Pokémon Showdown replays for Champions VGC 2026 Reg M-B, models the metagame, and produces `data/meta-usage.json`.

CHOMP (`../CHOMP`) — a separate but connected project. It answers one question at team preview: which four to bring and which two to lead. It reads ABRA's `meta-usage.json`. When ABRA improves, CHOMP gets better without a code change.

portfolio (`../portfolio`) — the public site presenting both.

The one principle that governs the data

Store raw, analyse on top. Every replay is archived whole in `data/games.ladder.raw-logs.jsonl`. A new question is a re-parse (`MODE=reparsed`), never a re-pull. Changing how games are segmented is a re-filter. Never design an analysis that requires re-fetching.

This paid off directly on 2026-07-24: measuring Champions' true paralysis rate needed the `|cant|` lines, which the parsed store does not keep. The raw archive had them.

3. Repository map

Project	Local	GitHub	Live
ABRA	C:\Users\willj\Projects\Pokemon\ABRA	willhoop/abra	—
CHOMP	C:\Users\willj\Projects\Pokemon\CHOMP	willhoop/chomp	runs locally in the browser
portfolio	C:\Users\willj\Projects\portfolio	willhoop/willhoop.github.io	GitHub Pages
HoopaDex	—	willhoop/hoopadex	willhoop.github.io/hoopadex
Event Desks	—	willhoop/event-desk	elitefourcapital.com

Warning: the root `index.html` of `willhoop/event-desk` IS the live site at elitefourcapital.com. Never push another project into that repository root.

4. Where things live in ABRA

engine/	the models and the ingest
data/	the store, the generated models, the browser data files
docs/	white paper, deck, technical docs, ADRs, this file
tests/	test suites
build/	generators (browser data, engine data, PDF omnibus)
web/	the site (index.html embeds a copy of the rollout engine)

The ingest and the store

- `engine/durable-ingest.js` — pull + store. **Source of truth for the schema;** `extract()` exported. `MODE=reparsed` rebuilds the store from the raw archive with no network.
- `data/games.ladder.jsonl` — the append-only store, 7,547 unique games, 3 days (22–24 July 2026).
- `data/games.ladder.raw-logs.jsonl` — every raw replay log, 41 MB. The reason re-parsing works.
- `engine/dedupe_store.py` — idempotent, order-preserving, atomic. Run it any time.

The quality filter — read this before using any game data

`data/quality-filter.json` is the single definition of a usable game. `engine/quality.js` and `engine/quality.py` are thin readers; neither hard-codes a threshold. `tests/test-quality.js` asserts both select an identical set of ids.

collected from Showdown	7547	(100.0%)	
after removing NAMED bot games	3378	(44.8%)	-4169
after removing accounts that behave like bots	1831	(24.3%)	-1547
after removing forfeits	1129	(15.0%)	-702
after removing games under 3 turns	1124	(14.9%)	-5
after requiring all four brought to be revealed	927	(12.3%)	-197

Only 927 games are usable. Two bot rules:

- Name** — `/^pctrlbot|bot\d|^[a-z]+bot$/i`. Catches 11 `pctrlbot<hex>` accounts playing 338–409 games each.
- Behaviour** — ≥ 50 games AND exactly one distinct team. This caught five accounts the name rule missed. `Carchdraw84172` played **459 games with one team, 367 in a single day**. Four of the five ran the SAME six Pokémon in 1,446 games. No account with ≥ 50 games has more than one team, so the rule separates cleanly.

Known limitation: both rules are floors, not proofs. Call the surviving set "no bot detected", never "human".

36 engines still bypass this filter. Only `analyze.js` and `chomp_ev.js` are wired. This is the largest outstanding correctness item — see section 8.

The models

Engine	What it does	Output
<code>analyze.js</code>	usage / bring / lead / win rates	<code>meta-usage.json</code> ← CHOMP reads this
<code>guru.py</code>	archetype matchup matrix	<code>guru-matchups.json</code>
<code>roles.py</code>	multi-label role taxonomy, 52+ roles, Wilson-gated	<code>pokemon-roles.json</code>
<code>nmf_roles.py</code>	emergent roles by matrix factorisation	<code>nmf-roles.json</code>
<code>war.py</code>	ridge-regularised RAPM (Wins Above Replacement)	<code>war.json</code>
<code>xatu.py</code> / <code>xatu_belief.py</code> / <code>xatu_context.py</code>	opponent set inference	<code>xatu-*.json</code>
<code>pory.py</code>	live win probability	<code>pory-eval.json</code>
<code>slowking*.py</code>	preview Nash equilibrium, playstyle	<code>slowking-*.json</code>
<code>chomp_ev.js</code>	does bring quality beat a coin?	<code>chomp-ev.json</code>
<code>medicham2-browser.js</code>	the doubles rollout engine	(used by others)
<code>champions_sim.js</code>	the official Showdown simulator	(new, see §6)
<code>illusion.js</code>	Zoroark detection by legality contradiction	<code>illusion.json</code>
<code>cores.js</code> , <code>playstyle.js</code> , <code>ditto.py</code> , <code>jolteon.py</code> , <code>counterplay.py</code> , <code>vocab.py</code>	supporting analyses	various
<code>sanity_check.py</code>	96 assertions across the whole system	—

5. What was found and fixed on 2026-07-24

The rollout engine was wrong in eight ways, every one silent

`engine/medicham2-browser.js` :

Defect	Effect
Status moves applied a random status	Thunder Wave burned a third of the time
Only Fake Out could flinch	Rock Slide's 30% and 32 others were inert
No status immunities	Fire types burned, Electric types paralysed
Priority a hand-typed table of 18 moves	All 14 negative-priority moves at 0; Trick Room (−7) at normal speed
Flinch never cleared at end of turn	A slow attacker's flinch stole the target's <i>next</i> turn
Intimidate unconditional −1	Blocked by 10 abilities; reversed by Defiant/Competitive — sign was wrong on the format's most-used ability
No powder immunity	Spore hit Grass types
Prankster hit Dark types	Illegal since Gen 7

Measured: **4.35 points** mean change in P(win) over 120 real matchups, max 24.2, favourite flipped in 9.2%. Pinned by `tests/test-rollout-effects.js` (39 assertions).

Five of the eight were found because the owner mentioned them in conversation. That is not a defect-detection process, and it is the argument behind ADR-001.

Other fixes

- **Nature table held 23 of 25.** Naughty (+Atk/−SpD) and Lax (+Def/−SpD) were absent and fell through to neutral. A missing row and a neutral row are indistinguishable by count, so the test now asserts the *direction* every stat moves.
- **The store's duplicate check read only the first 5,000 lines** while all 401 duplicates sat past line 7,144. Duplicates enter an append-only log at the END — the check was aimed away from the fault.
- `meta-usage.json` **was reporting the bot's team.** Old top six by usage: `garchomp`, `whimsicott`, `basculegion`, `kingambit`, `charizard`, `sylveon` — in full, the bot team's six. Corrected: `garchomp`, `incineroar`, `kingambit`, `sinistcha`, `whimsicott`, `basculegion`. Basculegion 34.1% → 17.9%. Whimsicott 31.9% → 17.9%. Charizard 29.1% → 16.5%.

Verified against the format's own source

Mechanic	Our engine	Champions <code>conditions.ts</code>	Measured from 7,948 raw logs
Full paralysis	12.5%	<code>randomChance(1, 8)</code>	13.8%, CI [11.9, 16.0]
Sleep, wake turn 2	33%	<code>sample([2, 3, 3])</code>	35.3%, CI [31.5, 39.2]
Freeze thaw / attempt	25%, forced at 3	<code>randomChance(1, 4), startTime = 3</code>	31.6%, CI [23.3, 41.4]

Stat formula confirmed from `scripts.ts`: `base + SP + 20`, HP `base + SP + 75`. Champions uses SP with a budget of 66, not EVs. Our `floor((2b+31)*50/100)+5+sp` reduces to exactly `b+20+sp`.

6. ADR-001 — the direction

`docs/ADR-001-use-the-champions-mod.md`. Status: **accepted**.

Showdown's `data/mods/champions/` implements this format exactly and is in the **master branch** (not the npm package — `pokemon-showdown@0.11.10` does not contain it). Format `gen9championsvgc2026regmb` is precisely ours.

- `engine/champions_sim.js` — runs it. Pinned to commit `20ad99ffc9a5a4a4e8fb56ab04ad8e4255b3f2b4`.
- `engine/prior_player.js` — ports our behaviour-clone policy into it.

With identical teams and identical policy, our engine and the official one disagree by 31.1 percentage points, flipping the favourite in 3 of 8 matchups. Everything fixed today was worth 4.35. The remaining gap is seven times larger than every repair combined.

Speed: official 29 battles/sec/core, ours 3,401 — 117× slower. Fine offline (927-game backtest = 7 min on 8 cores). Impossible live (CHOMP's budget is 50 ms). **Resolution: precompute matchup tables offline, ship the table.** The browser should not simulate at all.

The ADR records all four comparison attempts, including the three that produced wrong numbers — notably one where the policy port silently fell through to uniform random on 100% of decisions while reporting itself as a prior sampler.

7. Standards (from `docs/ARCHITECTURE.md`, v1.2)

Eleven standards, each paired with the fault that produced it and the check that enforces it. The ones that keep getting violated:

- **S1 — single source of truth.** Where multiple representations are unavoidable, one is definitive and every other is **generated by a script**, never hand-synchronised.
- **S2 — duplication that cannot be removed must be observable.** Consumer-driven contract test.
`CHOMP/tests/test-engine-contract.js` caught the mega drift on its first run.
- **S7 — the store has a shape, and it is tested.** No duplicate ids, `brought ≤ six`, `lead ≤ brought`, winner is one of the two players.
- **S8 — measured, never asserted.** No constant that affects a result is typed by hand.
- **S9 — golden master before refactoring.** `engine/validate_damage.js`, 31 scenarios vs `@smogon/calc`, run before and after every engine change.
- **S10 — enumerate closed domains, don't spot-check.** 25 natures, 18 types, 6 stat stages: walk the whole domain and assert direction. A spot-check cannot detect a missing row.
- **S11 — one publisher.** Exactly one process commits and pushes.

Living documentation — update in the SAME pass as any change

`docs/ABRA-whitepaper.md`, `docs/ABRA-deck-plain-english.md`, `docs/ABRA-technical-docs.md` (ASD-STE100, Diátaxis), `docs/SUMMARY.md`, `docs/MODELS.md`, `CHANGELOG.md` + version bump. Each with a matching PDF (`build/omnibus.py`, weasyprint). A number on the site must equal the number in the white paper and in the model's JSON report.

Changelog format

Keep a Changelog + SemVer, newest first, ISO dates. The top version MUST equal the version stamped on the project's primary artifact. Current: **3.1.1**.

8. Open work, priority order

1. **Repair section 1.** Nothing else matters first.
2. **Rewrite** `push-all.bat` — rebase, not `merge -X ours`.
3. **Wire the remaining 36 engines to the quality filter.** Every published number from GURU, WAR, roles, XATU, archetypes, playstyle, cores and the rest is computed over 7,547 games including 4,169 bot games. A patch converting the five `def load_games()` engines was written and NOT applied — the shape is identical across `war.py`, `roles.py`, `nmf_roles.py`, `vocab.py`, `counterplay.py`.
4. **Fix the site's claimed dataset size.** `data/live.js` says 7,716 games; `web/index.html` says "5,199 real Champions games". Both wrong. Lead with **927 clean**, show the funnel.
5. **Continue ADR-001** — golden master against the simulator, then the offline precompute job.

6. **Re-run every model** on the deduped, filtered store and report which published numbers move. Do not silently rewrite a prior conclusion; state what changed and why.
7. `web/index.html` **embeds a third copy of the rollout engine** and is not covered by the contract test. Largest remaining S1/S2 violation after the dex.

Standing older items

Cores + HodgeRank; per-model standalone docs; MEW self-play generator; Traylor/PokeAgent references.

9. How to work on this

- **Always be pushing.** Commit and push in the same pass as any change. Local-only is not done. Correct wording when it is: "written locally, not yet pushed".
- **Do not ask the owner to run commands.** You run them.
- **Never take over the machine.**
- **Three places must agree** — local files, GitHub, the live site — in the same pass.
- **Report negative results plainly.** Several today: preview roles tie a coin; better beliefs did NOT improve the bring decision (0.6940 vs 0.6931 for a coin).
- **Never quote a number without knowing which games it came from.**

Test suites

<code>python engine/sanity_check.py</code>	96 assertions, whole system
<code>node tests/test-rollout-effects.js</code>	39, rollout rules
<code>node tests/test-quality.js</code>	24, quality filter, JS/Python parity
<code>node engine/validate_damage.js</code>	31 scenarios vs @smogon/calc
<code>cd ../CHOMP</code>	
<code>node tests/test-engine-contract.js</code>	20, engine agreement
<code>node tests/test-mega-and-boosts.js</code>	28, megas, stages, all 25 natures
<code>node tests/test-engine.js</code>	4
<code>node tests/test-damage-golden.js</code>	16

All green as of 2026-07-24, on the store BEFORE the duplication incident. Re-run after the repair.

Running the official simulator

Needs a built master checkout — the champions mod is not in the npm package:

```
git clone --depth 1 https://github.com/smogon/pokemon-showdown
cd pokemon-showdown && npm install && node build
SHOWDOWN_PATH=/path/to/pokemon-showdown node engine/champions_sim.js
```

10. Honest limitations

- **The store spans 3 days.** No temporal design is possible. Any claim about meta drift is unsupported.
- **927 usable games** is a small sample. Many results will not clear zero on it, and that is the correct finding rather than a reason to loosen the filter.
- **Bot detection is a floor.** A bot playing under 50 games or varying its team escapes.
- `brought` **is what the replay REVEALED**, not what was selected. Requiring four conditions on game length, so bring statistics skew toward longer games. Say so when quoting them.

- **Sets are mostly unknown:** 1.6 of 4 moves revealed per set, 68% no item, 76% no ability. Anything that fills those gaps dominates a simulation result — this invalidated two engine comparisons before it was noticed.
- **The rollout engine disagrees with the official simulator by 31 points.** Until ADR-001 lands, treat every rollout-derived number as provisional.